prior to applying the migration to the database. For example, in later migrations, you may want to be able to execute some custom SQL as part of the procedure, such as bringing in non-seed data. I don't cover that in this book, but it is useful to know that the capability is available should you need it. However, I usually take a look at the migration code to ensure that the changes I intend to apply are reflected there. More than once, I've realized by looking at the migration code that I have not configured a property correctly. When that is the case, you can adjust the model configuration and then use the `remove-migration` (PowerShell) or `dotnet ef migrations remove` (CLI) commands to remove the existing migration before adding it back again, using the same command as before.

Having generated the migration and checked that it does what you want, you will apply it using one of the following commands:

```
[Powershell]
update-database
[CLI]
dotnet ef database update
```

Once you have executed one of these commands, you should see that a SQLite database file has been created in the Data folder: CityBreaks.db. Using whichever tool you prefer (I use the cross-platform DB Browser for SQLite; https://sqlitebrowser.org/), take a look at the schema (figure 8.6). In addition to tables for each of your model classes, the database includes a table named __EFMigrationsHistory.